

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Nguyễn Xuân Việt Đức - 22280012

Bài tập lý thuyết - Tuần 2

Câu 1: Chứng minh tính tối ưu của A* với heuristic nhất quán

Giả sử rằng thuật toán A* được dẫn dắt bởi một heuristic nhất quán sẽ tìm được đường đi tối ưu đến tất cả các nút đã mở rộng, tức là $g(n) = g^*(n)$ cho mọi nút mở rộng n .

- Giả định
Giả sử phản chứng rằng tồn tại một nút n sao cho $g(n) > g^*(n)$, tức là chi phí từ điểm xuất phát đến n theo đường đi A* không tối ưu.

- Xét nút n'
Vì n' là nút đầu tiên trên đường đi tối ưu đến n , ta có:

$$g(n') = g^*(n')$$

Do đó:

$$f(n') = g^*(n') + h(n')$$

- Sử dụng tính nhất quán của heuristic
Vì h là heuristic nhất quán, ta có bất đẳng thức:

$$h(n) \leq c(n', a, n) + h(n')$$

Do đó, vì $g^*(n) = g^*(n') + c(n', a, n)$, nên:

$$f(n') \leq g^*(n) + h(n)$$

Do giả định rằng $g(n) > g^*(n)$, chúng ta có:

$$f(n') < f(n)$$

Điều này dẫn đến mâu thuẫn với giả thiết rằng n là nút mở rộng đầu tiên, vì nếu $f(n') < f(n)$, thuật toán A* sẽ mở rộng n' trước n .

- Kết luận
Do mâu thuẫn xảy ra, giả định ban đầu $g(n) > g^*(n)$ không thể đúng. Do đó, với heuristic nhất quán, $g(n) = g^*(n)$ cho mọi nút mở rộng n , và thuật toán A* sẽ tìm được đường đi tối ưu.

Câu 2: A* search

- b : Hệ số phân nhánh, h : Độ sâu tìm kiếm

- Complete:
Hoàn chỉnh, trừ khi có vô hạn nốt với $f(n) \leq C^*$, với C^* là chi phí của lời giải tối ưu.
- Time: Trong trường hợp tệ nhất, độ phức tạp thời gian của A* là lũy thừa, với độ phức tạp

$$O(b^{h^*-h})$$

trong đó h^* là độ sâu của lời giải tối ưu.

- Space: Cần giữ tất cả các nốt trong bộ nhớ, gây ra độ phức tạp không gian cao.

$$O(b^{h^*})$$

- Optimal:
Tối ưu, nếu hàm heuristic $h(n)$ là nhất quán hoặc chấp nhận được. Ngoài ra, A* cũng đạt hiệu quả tối ưu vì nó mở rộng ít nốt nhất trong số các thuật toán sử dụng cùng một heuristic để tìm lời giải tối ưu. Việc mở rộng ít nốt hơn có thể bỏ qua lời giải tối ưu.

Câu 3: Chứng minh mọi công thức trên logic mệnh đề đều có thể đưa về dạng chuẩn tắc.

- Chuyển mọi phép toán về dạng cơ bản (phủ định, hội và tuyển):
Các phép toán như phép kéo theo ($\rightarrow$) và phép tương đương ($\leftrightarrow$) có thể được chuyển đổi thành các phép hội, tuyển và phủ định.

$$A \rightarrow B \equiv \neg A \vee B$$

$$A \leftrightarrow B \equiv (A \wedge B) \vee (\neg A \wedge \neg B)$$

Việc chuyển đổi này cho phép loại bỏ các phép toán phức tạp, chỉ giữ lại các phép hội (AND), tuyển (OR), và phủ định (NOT).

- Đưa phủ định vào trong:
Dựa trên **luật De Morgan** và các tính chất của phép phủ định, chúng ta có thể đẩy phủ định vào bên trong công thức sao cho phủ định chỉ áp dụng lên các biến mệnh đề (literal). Các quy tắc của De Morgan cho phép chúng ta thực hiện điều này:

$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$

$$\neg(A \vee B) \equiv \neg A \wedge \neg B$$

Điều này đảm bảo rằng tất cả các phủ định đều chỉ bao quanh các mệnh đề đơn, không phải các cụm mệnh đề phức tạp.

- Phân phối để các phép hội nằm ở ngoài cùng và các phép tuyển nằm ở bên trong:
Sau khi đơn giản hóa và đưa phủ định vào trong, chúng ta cần phân phối các phép hội qua các phép tuyển để đưa công thức về dạng CNF. Cụ thể, chúng ta áp dụng quy tắc phân phối sau:

$$A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$$

Bằng cách này, chúng ta có thể đảm bảo rằng cấu trúc của công thức là một phép hội của các tuyển (conjunction of disjunctions), là dạng chuẩn của CNF.

- Bảo toàn ngữ nghĩa:
Trong suốt quá trình chuyển đổi, mỗi bước đều bảo toàn ngữ nghĩa của công thức gốc, tức là không làm thay đổi tập các mô hình (các cách gán giá trị đúng/sai) của công thức. Ví dụ: nếu công thức ban đầu là đúng trong một mô hình cụ thể, thì công thức sau khi chuyển đổi cũng sẽ đúng trong cùng mô hình đó.
- So sánh với các quy tắc suy diễn tổng quát:
Việc chuyển đổi sang CNF chỉ đơn giản hóa cấu trúc công thức mà không làm tăng số lượng công thức, như đã mô tả. Điều này khác với các quy tắc suy diễn, trong đó số lượng công thức có thể tăng đáng kể do quá trình suy luận.
Do đó, việc chuyển đổi sang CNF không dẫn đến "bùng nổ" về số lượng công thức và giúp đơn giản hóa việc suy luận, tương tự như kỹ thuật AC-3 trong tìm kiếm backtracking.
- Kết luận:
Từ các bước trên, ta có thể kết luận rằng bất kỳ công thức nào trong logic mệnh đề đều có thể chuyển về dạng chuẩn tắc liên hợp (CNF).